

Llama

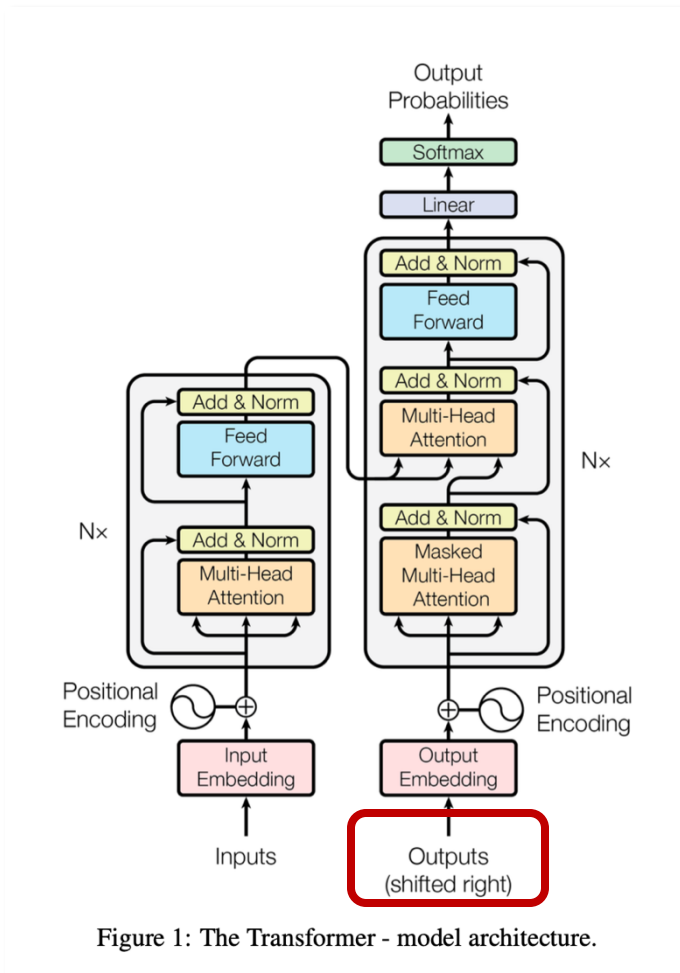
论文带读+手撕代码

LLM实战速成班

1

Transformer回顾

1.1 分词 注：（可以用字分词也可以用词）



小狗和小猫玩



小	狗	和	小	猫	玩
---	---	---	---	---	---

1.2 向量化

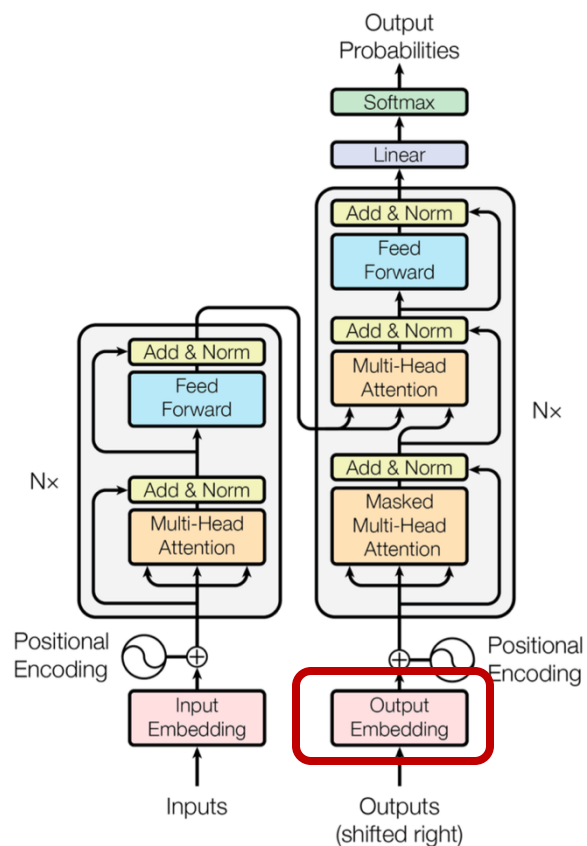


Figure 1: The Transformer - model architecture.

查ID

		Input Tokens						
		小	狗	和	小	貓	玩	
	t	0	1	2	3	4	5	
狗	0	0	1	0	0	0	0	0
貓	1	0	0	0	0	1	0	0
牛	2	0	0	0	0	0	0	0
大	3	0	0	0	0	0	0	0
中	4	0	0	0	0	0	0	0
小	5	1	0	0	1	0	0	0
吃	6	0	0	0	0	0	0	0
玩	7	0	0	0	0	0	0	1
跑	8	0	0	0	0	0	0	0
和	9	0	0	1	0	0	0	0
你	10	0	0	0	0	0	0	0
我	11	0	0	0	0	0	0	0
他	12	0	0	0	0	0	0	0
的	13	0	0	0	0	0	0	0
好	14	0	0	0	0	0	0	0
在	15	0	0	0	0	0	0	0

乘embedding矩阵

Word Embedding

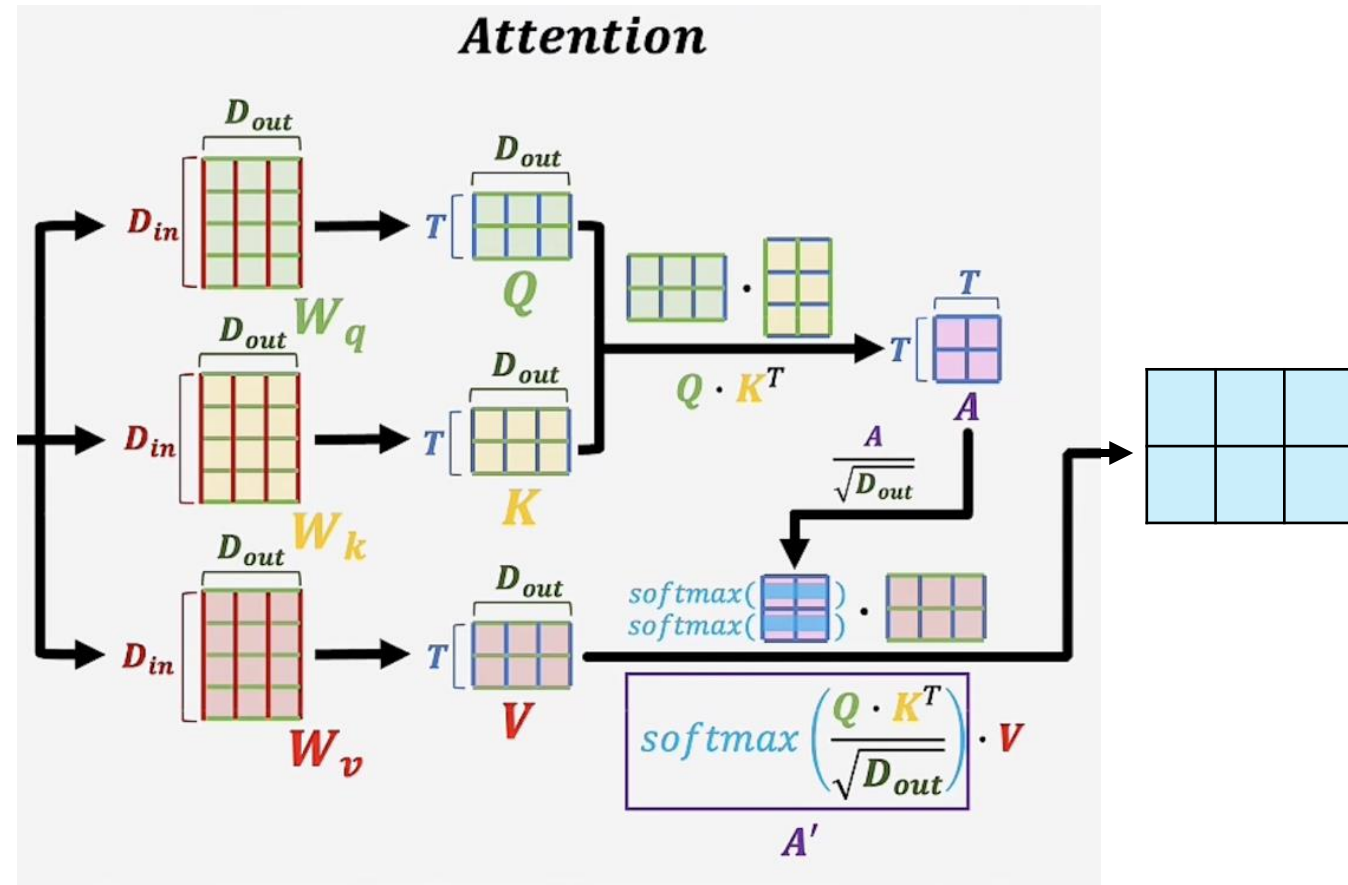
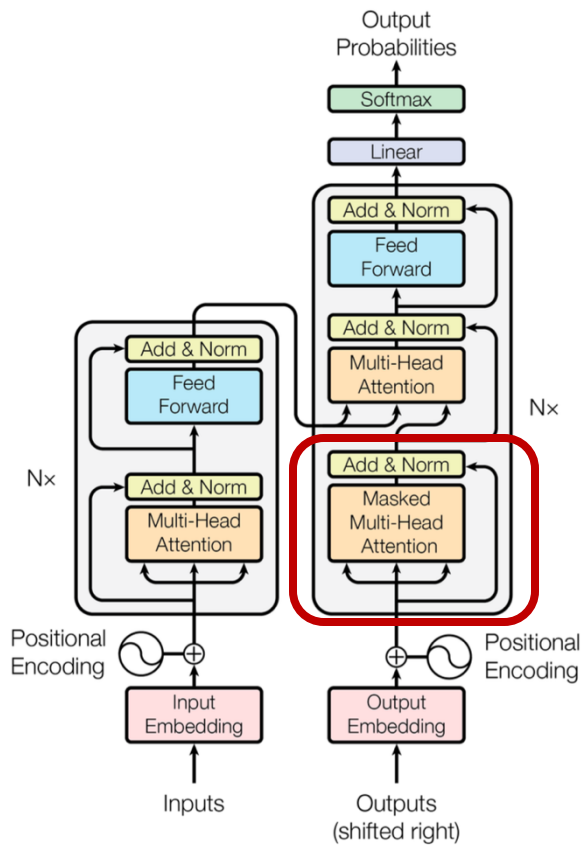
狗	猫	牛	大	中	小	吃	玩	跑	和	你	我	他	的	好	在
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3	0	2	1	2	3	2	2	3	0	1	0	0	0	0	1
0	3	0	3	0	2	2	1	1	3	0	1	1	0	1	0
0	1	1	2	3	1	1	3	0	1	1	0	1	1	0	1
1	1	3	3	3	3	2	1	3	2	1	1	0	1	1	0
3	1	0	3	0	2	3	2	2	2	0	0	0	1	0	0

→

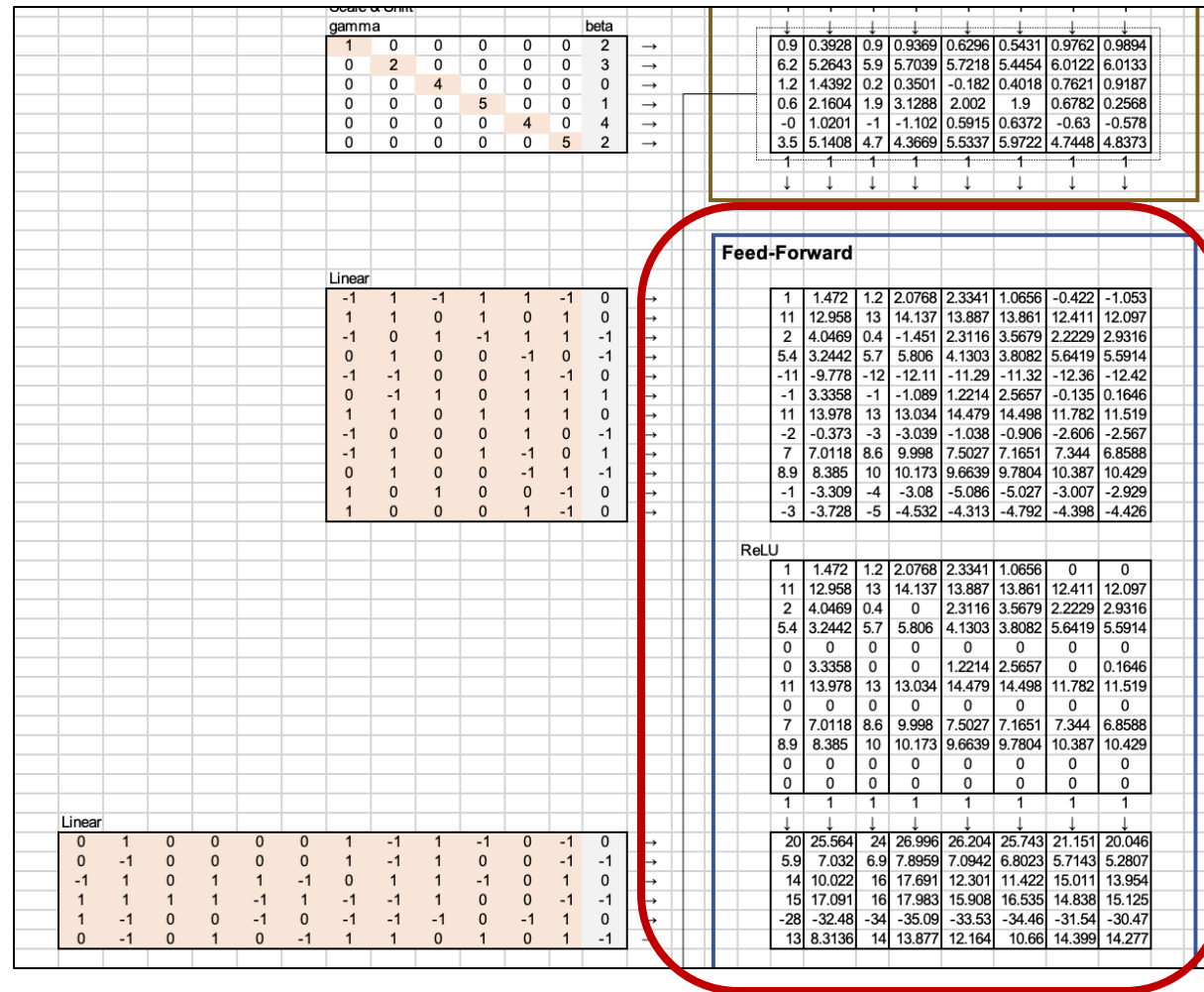
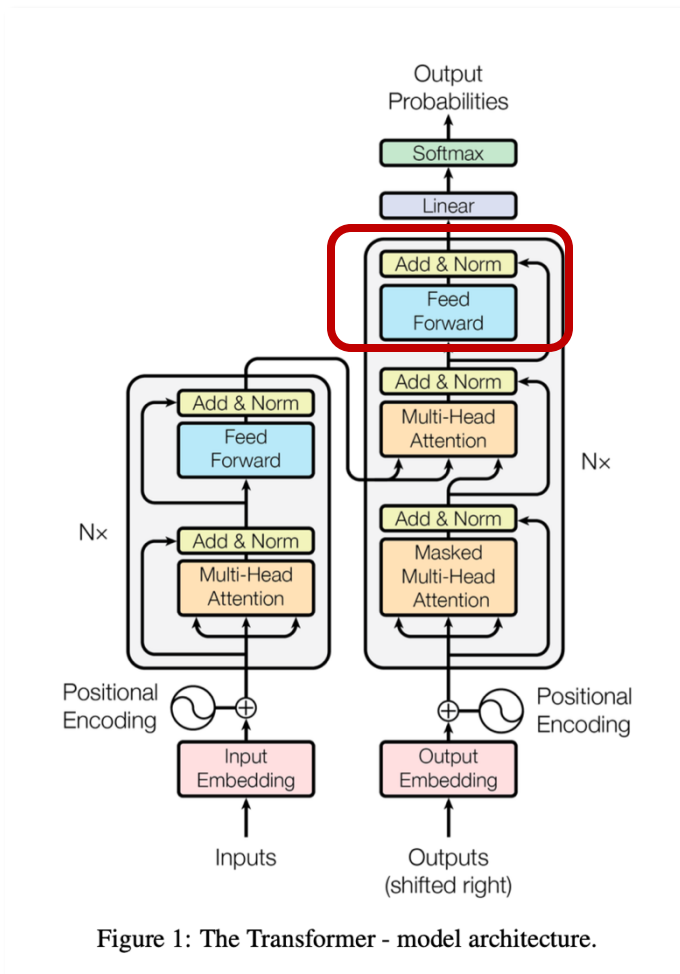
3	3	0	3	0	2
2	0	3	2	3	1
1	0	1	1	1	3
3	1	2	3	1	1
2	3	2	2	1	2

↓

1.4 Attention



1.5 FFN



1.6 预测下一个token

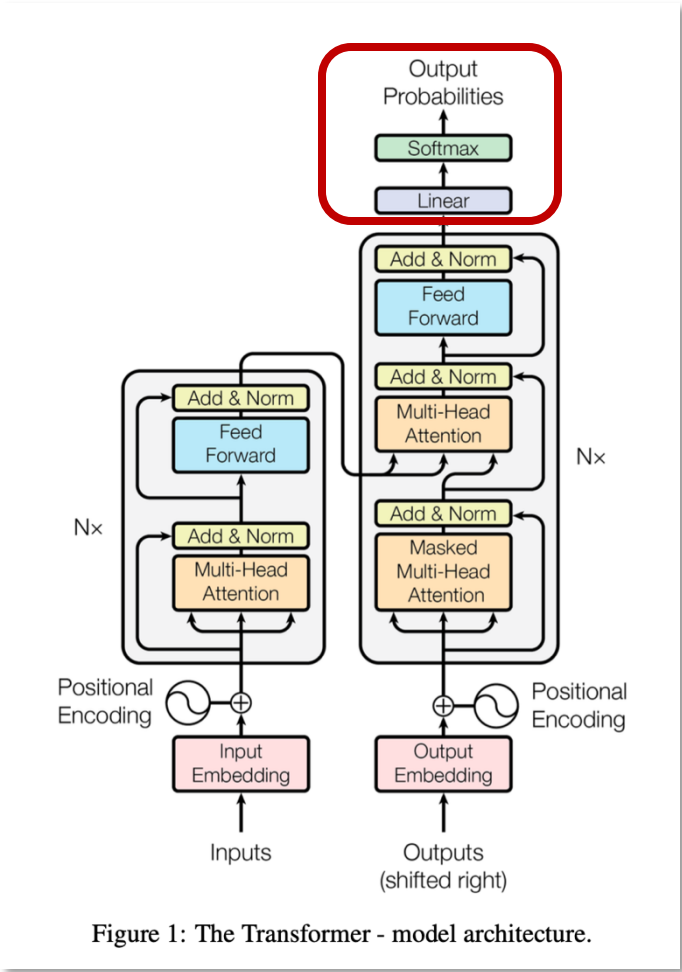


Figure 1: The Transformer - model architecture.

	t	0	1	2	3	4	5	6	7
[S]	0						0.0213		
a	1						9E-06		
and	2						0.0001		
apples	3						0.0265		
brings	4						0.0074		
by	5						0.0042		
cat	6						3E-06		
day	7						2E-07		
dog	8						4E-07		
eat	9						0.0019		
fill	10						0.6217		
friend	11						4E-07		
happy	12						0.0011		
house	13						4E-07		
laughter	14						3E-05		
light	15						0.1167		
music	16						6E-07		
night	17						2E-06		
play	18						0.0445		
quiet	19						6E-06		
smile	20						0.0977		
the	21						0.0569		
to	22						2E-10		
water	23						0.0001		

2

Llama3论文精读

2.1 SwiGLU

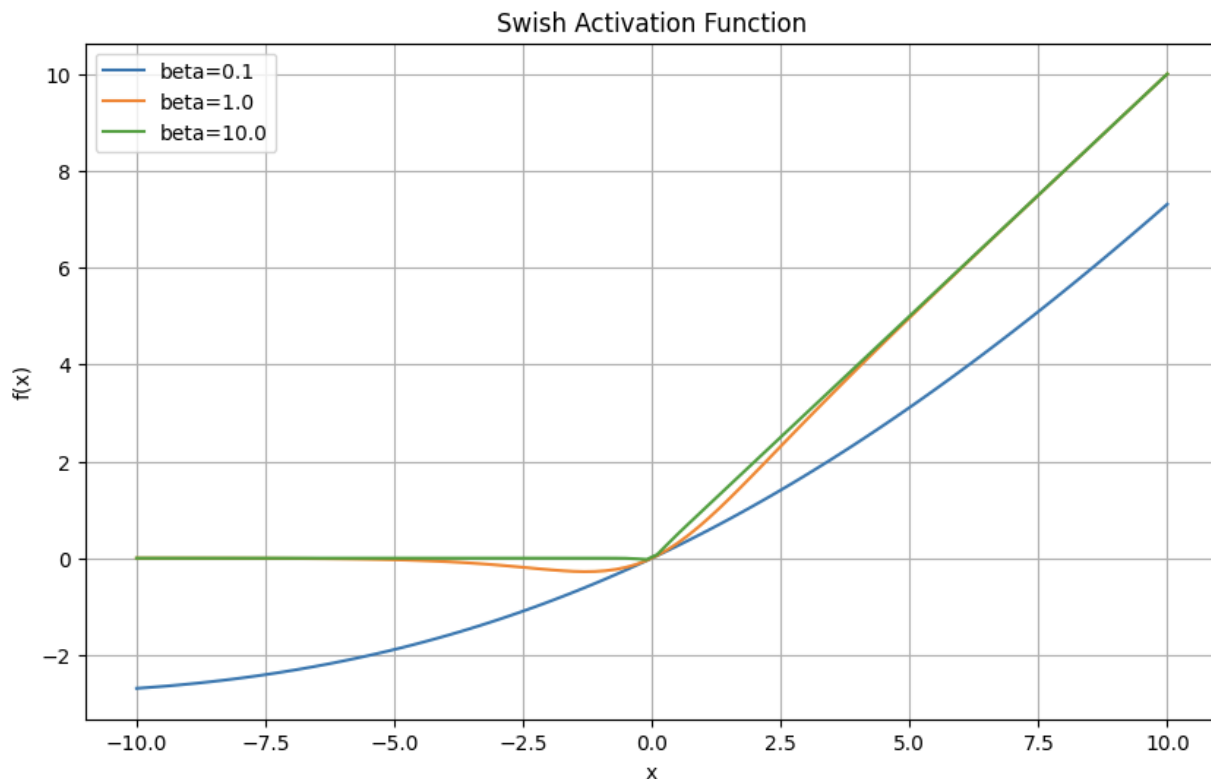


Table 2: GLUE Language-Understanding Benchmark [Wang et al., 2018] (dev).

	Score Average	CoLA MCC	SST-2 Acc	MRPC F1	MRPC Acc	STS-B PCC	STS-B SCC	QQP F1	QQP Acc	MNLI-m Acc	MNLI-mm Acc	QNLI Acc	RTE Acc
FFN _{ReLU}	83.80	51.32	94.04	93.08	90.20	89.64	89.42	89.01	91.75	85.83	86.42	92.81	80.14
FFN _{GLU}	83.86	53.48	94.04	92.81	90.20	89.69	89.49	88.63	91.62	85.89	86.13	92.39	80.51
FFN _{Swish}	83.60	49.79	93.69	92.31	89.46	89.20	88.98	88.84	91.67	85.22	85.02	92.33	81.23
FFN _{GLU}	84.20	49.16	94.27	92.39	89.46	89.46	89.35	88.79	91.62	86.36	86.18	92.92	84.12
FFN _{GeLU}	84.12	53.65	93.92	92.68	89.71	90.26	90.13	89.11	91.85	86.15	86.17	92.81	79.42
FFN _{tanhmax}	83.79	51.02	94.38	92.28	89.46	90.06	89.84	88.95	91.69	86.90	87.08	92.92	81.95
FFN _{SwiGLU}	84.36	51.59	93.92	92.23	88.97	90.32	90.13	89.14	91.87	86.45	86.47	92.93	83.39
FFN _{ReLU}	84.07	50.16	94.38	92.06	89.22	89.97	89.85	88.86	91.72	86.29	86.40	92.08	81.99

模型架构

• **基础架构**: 基于标准的 **Transformer** 架构，但进行了优化：

- 使用 **RMSNorm** (Root Mean Square Layer Normalization) 替代传统 LayerNorm，提升训练稳定性。
- **SwiGLU** 激活函数取代 ReLU，增强非线性表达能力。
- **RoPE** (Rotary Positional Embedding) 位置编码，在注意力层中动态生成位置信息。

2.2 RMS Norm

1. LayerNorm

LayerNorm (Layer Normalization) 规范化是对每个样本的特征维度进行规范化。其主要思想是对每个样本的所有特征进行标准化，使得每个样本在其特征维度上具有相同的均值和标准差。

公式：对于输入向量 $x \in \mathbb{R}^d$ ，LayerNorm的计算如下：

$$\hat{x}_i = \frac{x_i - \mu}{\sigma + \epsilon} \cdot \gamma + \beta$$

其中：

- $\mu = \frac{1}{d} \sum_{i=1}^d x_i$ 是输入的均值。
- $\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$ 是输入的标准差。
- γ 和 β 是可学习的缩放和平移参数。
- ϵ 是一个小常数，避免除以零。

2. RMSNorm

RMSNorm (Root Mean Square Normalization) 是LayerNorm的一个变体，它没有计算均值，而是仅通过根均方差（RMS）对输入进行规范化。RMSNorm的主要目标是减少计算复杂度，因为不需要计算每个样本的均值，只需要计算标准差。

公式：RMSNorm的计算如下：

$$\hat{x}_i = \frac{x_i}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma + \beta$$

其中：

- 计算根均方差： $\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$ 。
- γ 和 β 是可学习的缩放和平移参数。
- ϵ 是一个小常数，避免除以零。

举例：

假设有一个输入向量 $x = [1.0, 2.0, 3.0]$ （其中 $d = 3$ ）。

LayerNorm:

1. 计算均值： $\mu = \frac{1+2+3}{3} = 2.0$ 。
2. 计算标准差： $\sigma = \sqrt{\frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3}} = \sqrt{\frac{2}{3}} \approx 0.8165$ 。
3. 归一化后：

$$\hat{x} = \frac{x - \mu}{\sigma + \epsilon} = \frac{[1.0, 2.0, 3.0] - 2.0}{0.8165 + \epsilon} = \frac{[-1.0, 0.0, 1.0]}{0.8165 + \epsilon}$$

4. 缩放和平移：乘以 γ 和加上 β 。

RMSNorm:

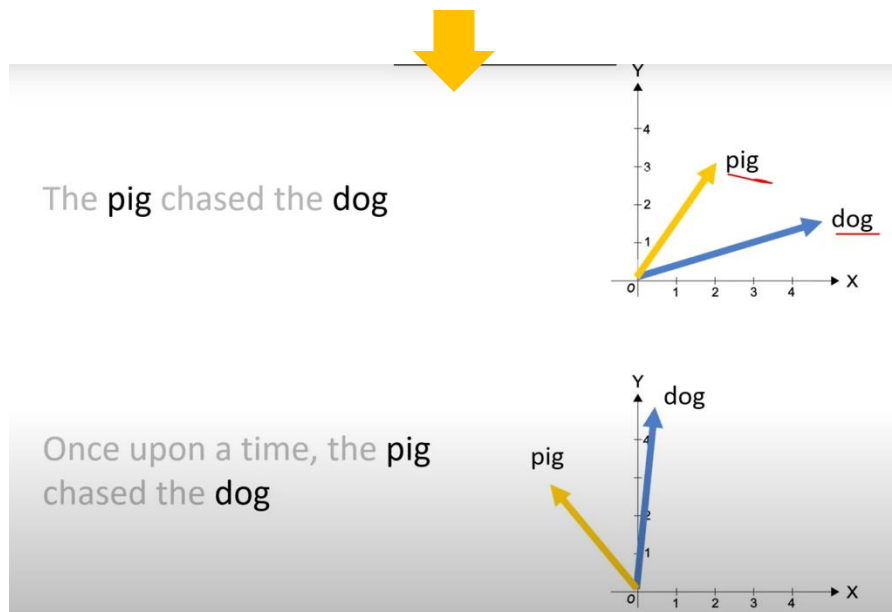
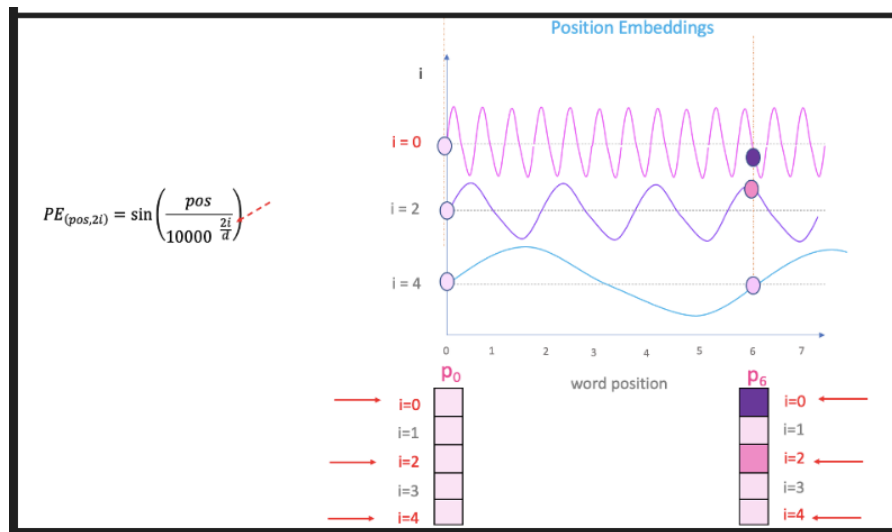
1. 计算根均方差： $\text{RMS}(x) = \sqrt{\frac{1^2 + 2^2 + 3^2}{3}} = \sqrt{\frac{14}{3}} \approx 2.160$ 。
2. 归一化后：

$$\hat{x} = \frac{x}{\text{RMS}(x) + \epsilon} = \frac{[1.0, 2.0, 3.0]}{2.160 + \epsilon}$$

3. 缩放和平移：乘以 γ 和加上 β 。

RMSNorm的计算更简单，减少了均值计算的开销，加速了推理和训练。

2.3 RoPE



为什么要用旋转位置编码?

我	喜欢	打	篮球
他	早上	吃	早饭

	t	0	1	2	3	4	5
d							
0		0.9217964	0.987	0.998	1	1	1
1		0.7147135	0.312	0.126	0.05	0.02	0.008
2		0.3676445	0.889	0.982	0.997	1	1
3		0.999766	0.592	0.25	0.1	0.04	0.016
4		-0.4075226	0.702	0.951	0.992	0.999	1

3.922	3.987	0.998	4	1	3
2.715	0.312	3.126	2.05	3.02	1.008
1.368	0.889	1.982	1.997	2	4
4	1.592	2.25	3.1	1.04	1.016
1.592	3.702	2.951	2.992	1.999	3

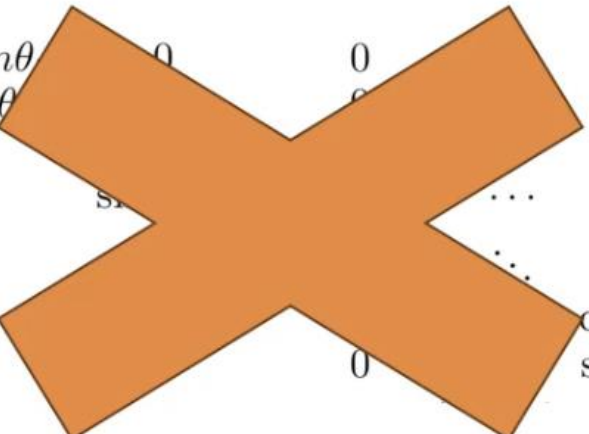
$$\mathbf{PE}_i^{(2k)} = \sin\left(\frac{i}{10000^{2k/d}}\right), \quad \mathbf{PE}_i^{(2k+1)} = \cos\left(\frac{i}{10000^{2k/d}}\right)$$

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

2.3 RoPE

我 | 喜欢 | 打 | 篮球
他 | 早上 | 吃 | 早饭

怎么解释？

$$R_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & 0 & 0 \\ \vdots & \vdots & 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & 0 & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$


$$R_{\Theta, m}^d \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}$$

- **更好的长序列处理能力：** RoPE能够有效捕捉长序列中各个位置之间的相对关系，并且能够在推理时较好地处理长序列的泛化问题。
- **灵活性和可扩展性：** RoPE可以轻松地扩展到比训练时更长的序列，而传统的绝对位置编码方法在遇到比训练时长的序列时表现较差。
- **更强的表达能力：** 通过旋转的方式，RoPE能够提供更加丰富的位置信息，使得模型能够更好地学习不同位置间的关系。

2.4 GQA

Group Query Attention (GQA) 机制详解

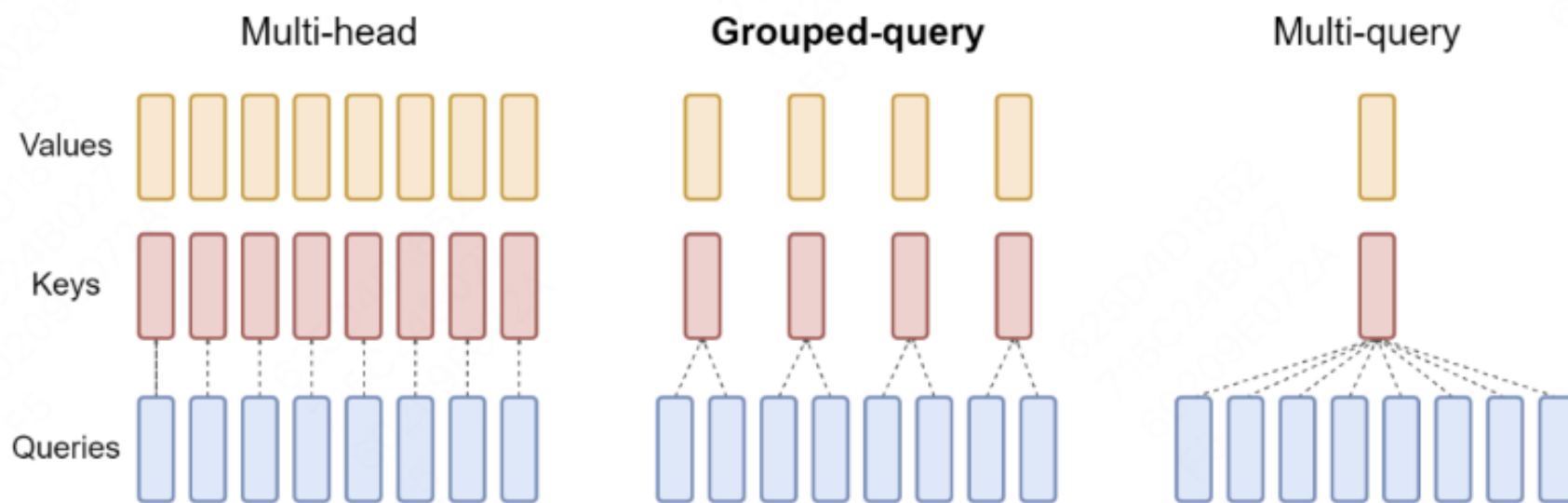


Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

2.5 训练过程

